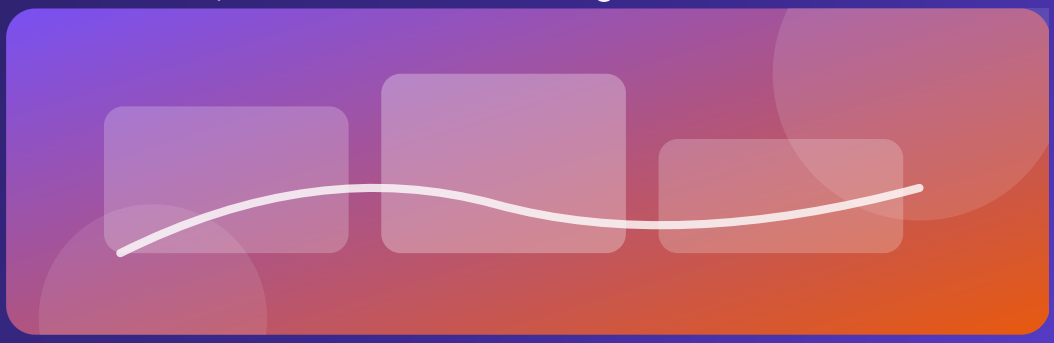
SkillNavora · Product Management Master Handbook

Module 12: Agentic AI and AI Systems

Detailed supplement: automation versus agents, n8n workflows, the layers of an AI system, AI economics, monetisation, evaluation and working with AI teams



Written by Varun Kumar, for SkillNavora · Draft

Official site: www.skillnavora.in

Prepared for Skillnavora — Learn. Practise. Implement. Grow.

MODULE 12 · HOW TO USE THIS SUPPLEMENT

Coverage map: what is new and what is already in the module

The main Module 12 handbook covers agents in depth. This supplement adds the deck's practical layers: automation versus agents, n8n flows, the layers of an AI system, the unit economics of AI features, monetisation patterns, a cost versus value matrix, evaluation, and working with AI and ML teams.

Topic (Week 11 deck)	Handbook section	Status here
Automation versus agent	12.x (partial)	Extended: a table with examples
n8n: trigger, node, workflow, JSON	—	New: core concepts with PM analogies
Build a feedback-to-Slack flow	—	New: the four-step demo flow
Layers of an AI system	12.x (partial)	New: five layers, with a PM question each
AI economics: tokens, infrastructure, margin	—	New: how cost is built up, with the deck's figures
Monetisation patterns and cases	—	New: add-on, per-action and per-resolution pricing
Bundle or charge for an AI feature	—	New: a usage rule of thumb
Cost versus value matrix	—	New: kill, re-evaluate, ignore, scale
Evaluating AI systems	12.x (partial)	Extended: metrics, tests and PM actions
Working with AI and ML teams	—	New: what to agree early

NOTE ON THE SOURCE

The deck's prices, revenue figures and case-study percentages are examples quoted in the course. They change over time and some are company claims. Check each one on the company's own page before you quote it.

Automation versus agent

An automation runs fixed steps. An agent decides what to do next, within limits. Choose automation when the steps are known; choose an agent when the path depends on the input.

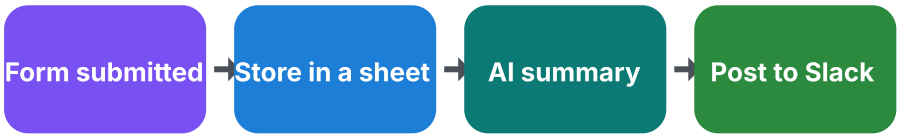
Type	What it does	Example
Automation	Follows fixed steps	Send a Slack alert when the NPS score is below 7
Agent	Decides what to do	Route a message to the right team based on its topic

NEW · WORKFLOWS

n8n: triggers, nodes and JSON

n8n is an open-source, node-based workflow tool. Think of it as a blueprint for a process, built from connected steps.

Term	Meaning	PM analogy
Trigger	Starts the workflow	An event happens
Node	One action step	A task in your flow
Workflow	The full sequence	A process blueprint
JSON	Structured data: key-value pairs; each node passes it on	The data a step hands to the next one

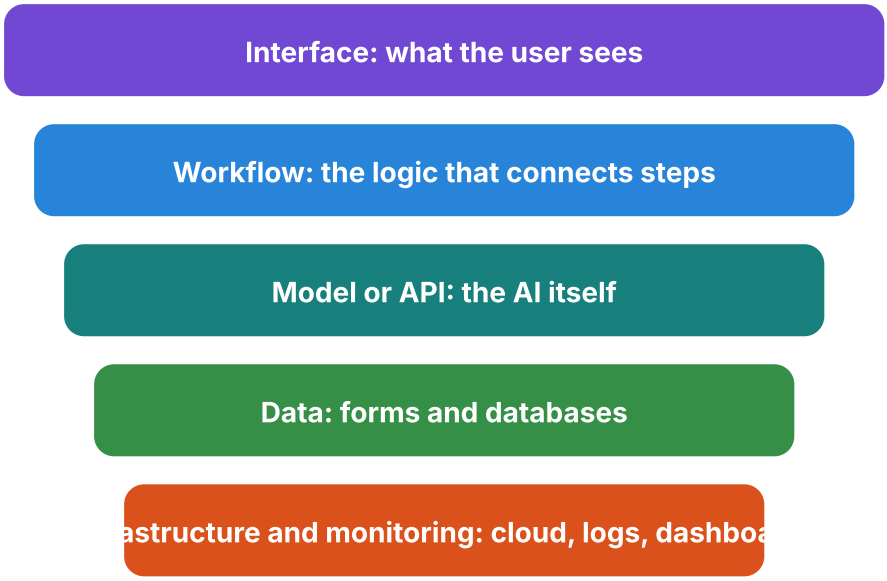


Feedback-to-Slack demo flow

NEW · LAYERS

The five layers of an AI system

An AI feature is more than a model. It is a stack of layers, each with its own cost and failure modes.



The layers of an AI system

NEW · AI ECONOMICS

Tokens, infrastructure and margin

The cost of an AI feature is built from tokens, models, infrastructure and margin. A simple form: **API cost = price per token × tokens used**. Margins depend on volume and on how often outputs are reused.



Illustrative cost split for one AI feature

Deck example (verify current prices): at the rates quoted in the deck, one million input tokens cost about $5 and one million output tokens about $15. The same logic applies to any provider: count the tokens each user action uses, then multiply.

NEW · MONETISATION

Patterns and the bundle rule

The deck groups AI monetisation into patterns. Each one needs a clear unit of value.

Pattern	Example in the deck	How it charges
Add-on subscription	An AI add-on priced per month	Monthly fee on top of the product
Per action	An AI writing tool with templates	Charge per generated output
Per resolution	An AI support agent	Charge only when a ticket is resolved
Bundled	A free assistant inside a tool people already use	No separate charge; it drives retention

Bundle or charge (usage rule of thumb): if fewer than 70 percent of users use the feature, charge for it separately; if more than 70 percent use it, bundle it to drive stickiness. Test the threshold with your own data.

NEW · DECISIONS

Cost versus value: kill, re-evaluate, ignore, scale

Plot each AI feature by its cost and its value. The matrix gives one of four decisions.



Cost versus value matrix

Re-evaluating a costly feature usually means changing the model, the prompt length or how often the feature runs.

EXTENDED · EVALUATION

Evaluate AI systems with a small test set

Automation is not impact. Evaluation links AI performance to business results, detects drift and waste, and justifies return on investment.

Metric	How to test	PM action
Accuracy	Check 10 sample outputs	Tune the prompt or the feedback loop
Latency	Average response time	Optimise the slowest node
Cost	Token logs	Estimate return on investment
Bias	Compare results across segments	Document the risk
Hallucination	Fact-check a subset	Improve retrieval

The deck's case figures for AI coding and writing tools are company claims, quoted as examples. Verify them before using them in a plan.

NEW · COLLABORATION

Working with AI and ML teams

AI and ML teams work best when the PM agrees on four things early:

- The input and the output of the feature.
- The prompts, shared with the developers who build them.
- The logs and costs, tracked together.
- What counts as success, and when to stop.

Quiz with answers

1. When is an agent a better choice than an automation?
   Answer: when the path depends on the input and the steps cannot be fixed in advance.
2. What is a node in n8n?
   Answer: one action step in a workflow.
3. What is the formula for the API cost of a feature?
   Answer: price per token multiplied by the tokens used.
4. In the matrix, what should you do with a costly feature of low value?
   Answer: kill it.
5. What is the 70 percent rule of thumb?
   Answer: if fewer than 70 percent of users use the feature, charge for it; if more, bundle it.

MODULE 12 · FINAL

Completion checklist

Completion checklist

Done	Item
☐	Read the coverage map
☐	Built one n8n flow with a trigger, a node and an output
☐	Mapped the five layers for one AI feature
☐	Calculated the token cost of one user action
☐	Placed one feature on the cost versus value matrix
☐	Wrote an evaluation table with three metrics

MODULE 12 · CLASS SLIDES

Agentic AI and AI Systems

Varun Kumar · SkillNavora

Prepared for Skillnavora · Learn. Practise. Implement. Grow.

Product Management Roadmap

PHASE 1
CRAFT
Modules 1–4

1. Fundamentals of Product Management
2. Product Strategy & Strategic Thinking
3. Problem Discovery, Market & User Research
4. Problem Framing & Prioritization

PHASE 2
CRAFT + AI
Modules 5–8

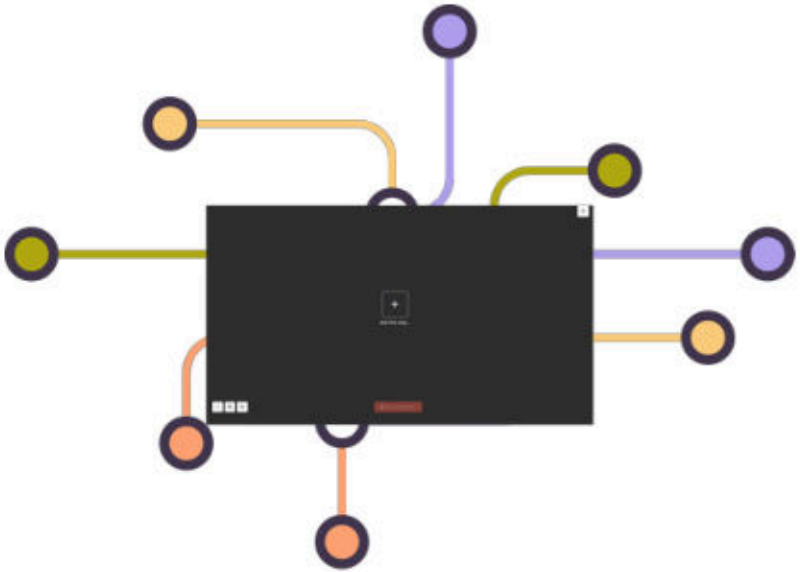
5. UX & Design Collaboration
6. Effective Communication as a PM
7. Analytics & Metrics
8. Product Launch, Growth & Monetization

PHASE 3
DEEP AI
Modules 9–13

9. Tech 101 & System Design
10. LLMs, RAGs & AI Foundations
11. Vibe Coding for PMs
12. Agentic AI & AI Systems in Production — WEEK 12
13. Capstone Project & Interview Prep

AI Systems in Action — Automate What Matters

“Where ideas start running on their own.”



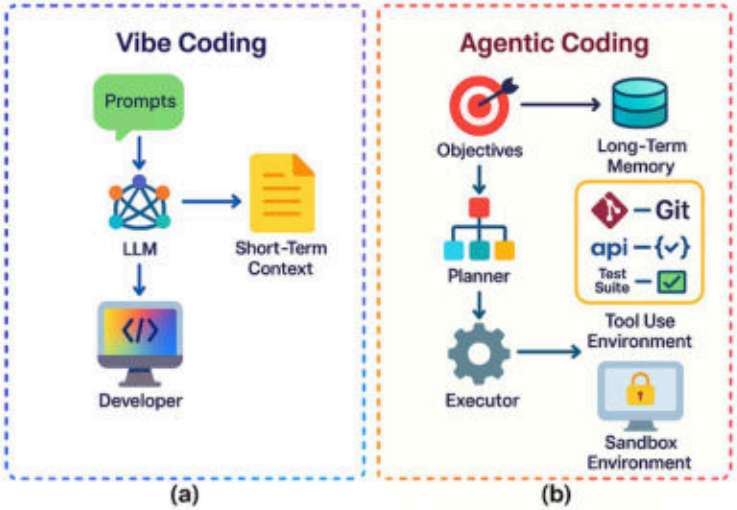
Agenda for next 4 sessions

- Day 1: Intro to vibe coding, system prompts, vibe coding tools and best practices
- Day 2: Working with MCPs, Building a small Lovable app, getting your kit ready for Day 3
- **Day 3: Building a full webapp on Antigravity, deploying on Netlify and uploading on Github**
- Day 4: Tokenomics and guide to building your own portfolio, workflow automation with n8n

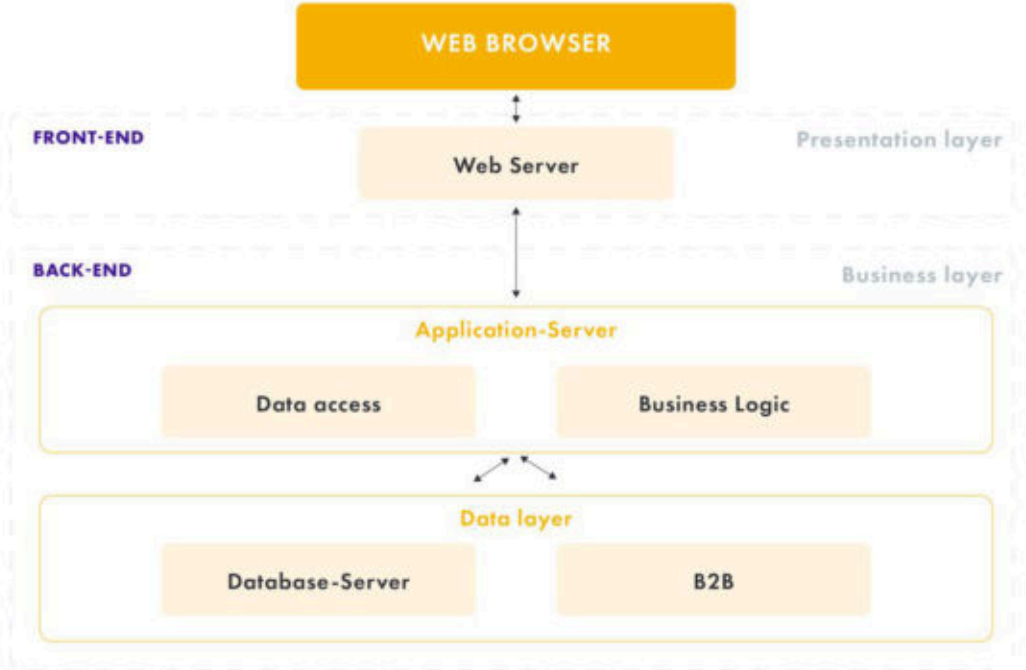
~~Vibe Coding~~

Agentic Coding

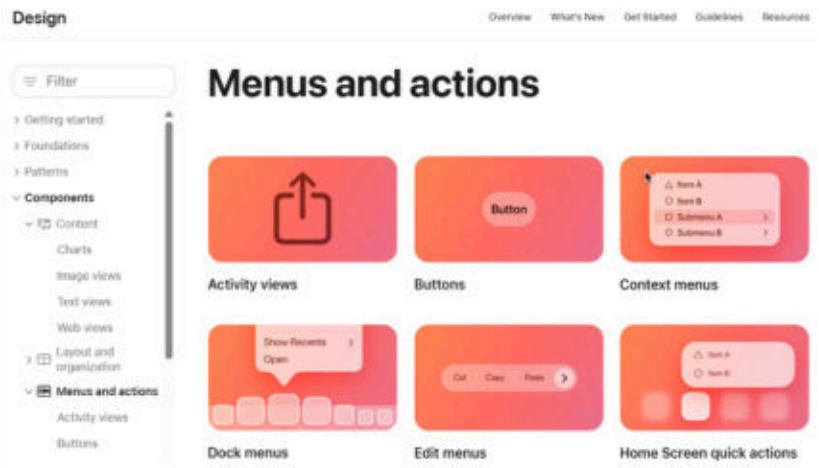
Full Stack



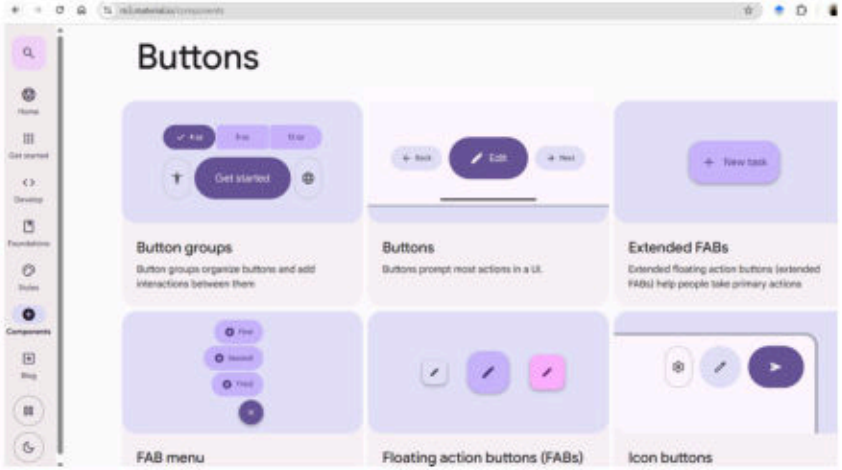
How does your website work?



Essentials : Design



https://developer.apple.com/design/human-interface-guidelines/menus-and-actions



https://m3.material.io/components

And where to get screenshots?

Any website/tool/app/product you find similar to what you want to build

- Chatbots? Think ChatGPT, claude, (even lovable itself!)
- Social media? Shamelessly throw in Linkedin, insta, etc
- Forums? Think Reddit, Quora, etc

Other resources (for “inspiration”):

1. https://dribbble.com/
2. https://21st.dev/community/components
3. https://www.behance.net/search/projects

Read more about Lovable best practices here: https://lovable.dev/blog/2025-01-16-lovable-prompting-handbook

A little bit about Databases



Live building an e-commerce app

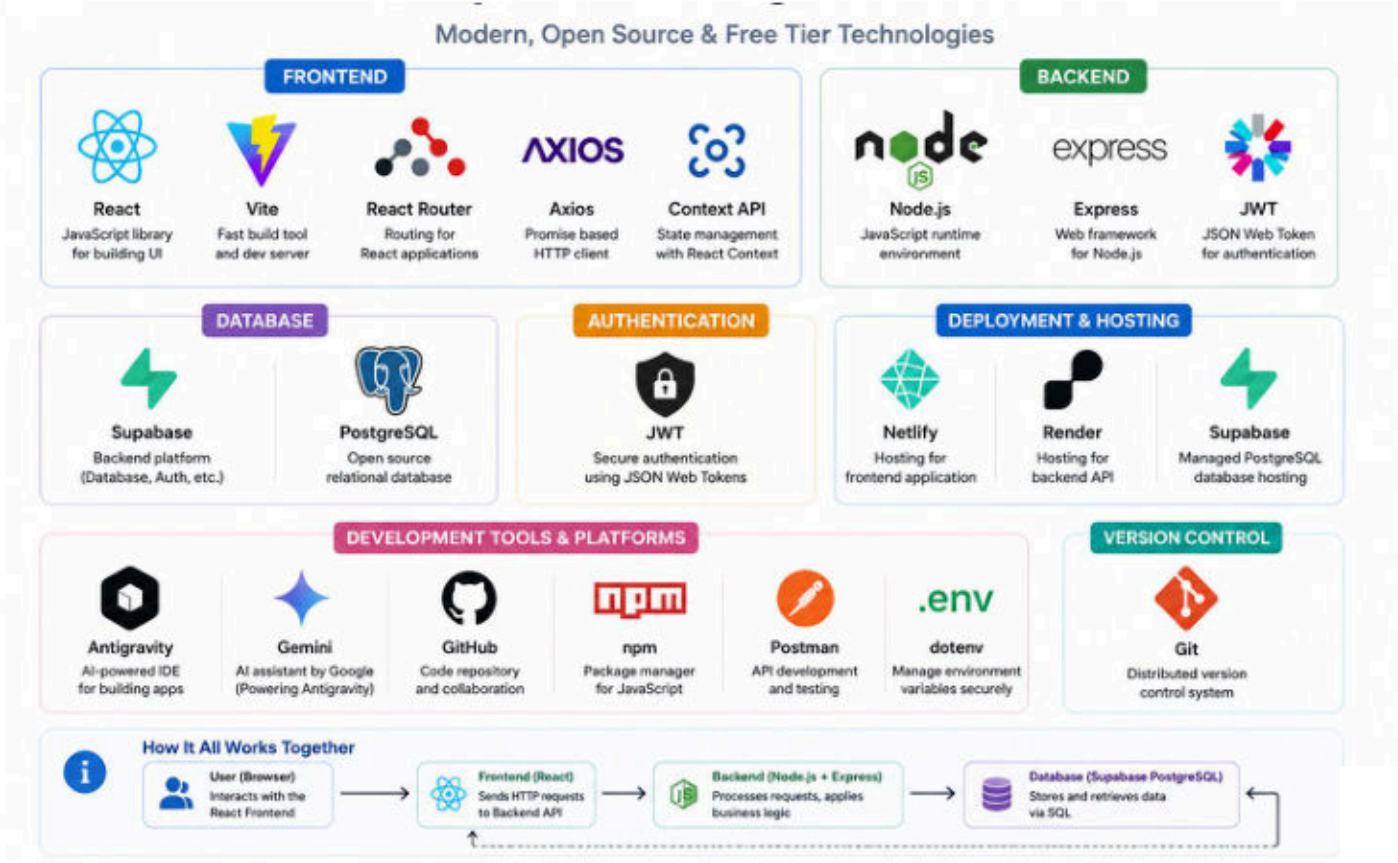
A small local business wants to launch its first online store but cannot afford expensive ecommerce platforms.

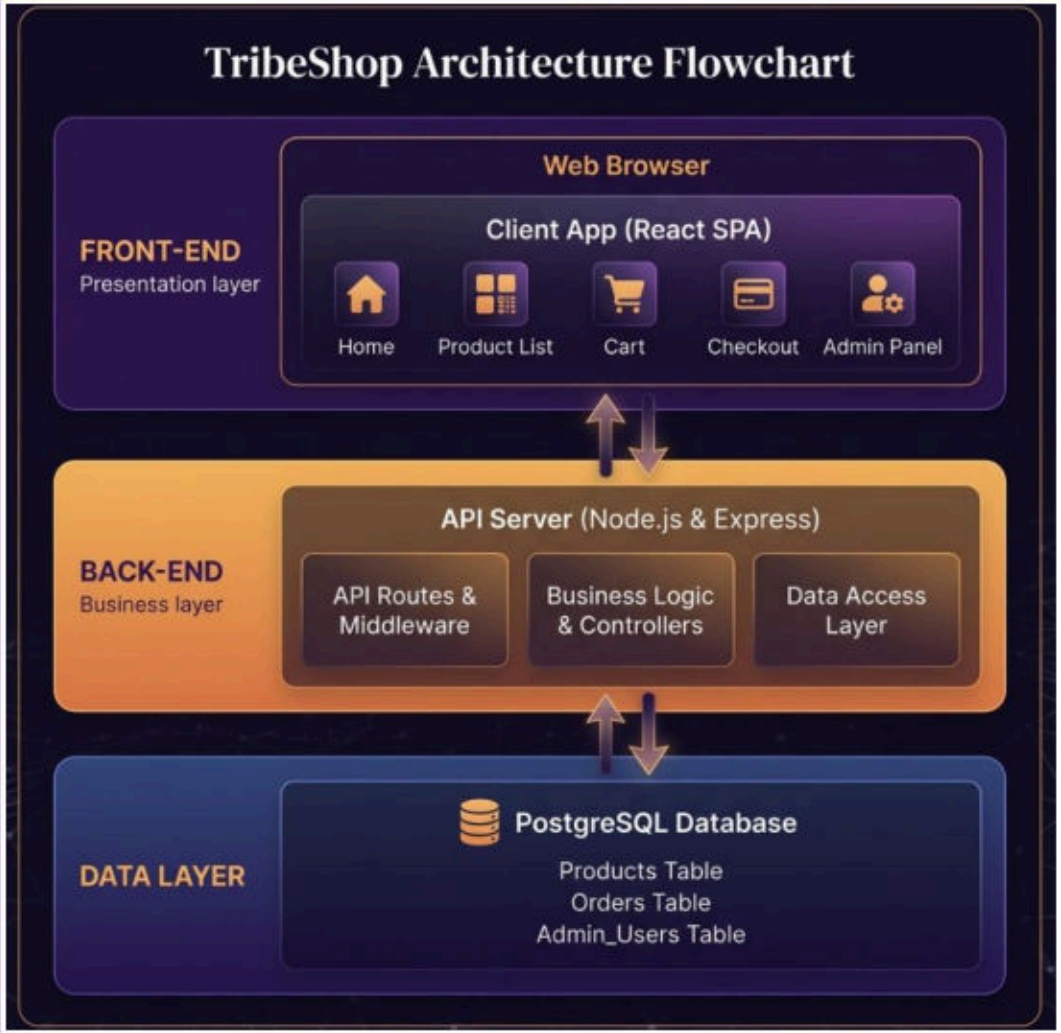
They need a lightweight, easy-to-manage web application that allows customers to browse products and place orders, while enabling administrators to manage inventory and track orders efficiently.

Your task is to design and develop a production-ready ecommerce solution called TribeShop using modern web technologies and free deployment tools.

Tech we are using

https://github.com/sssaloni/tribeshop





https://github.com/sssaloni/tribeshop

Airtribe

TribeShop – End-to-End Full Stack Vibe Coding Workflow

From Setup to Running – Phase 0 to Phase 17

0 Phase 0: Create Accounts

Create accounts on:

- GitHub
- Supabase
- Render
- Netlify
- Antigravity IDE

Download:

- Antigravity IDE
- GitHub Desktop

1 Phase 1: Create Repository

GitHub

Create a new repository

Repository name: tribeshop

Description (optional): TribeShop – E-commerce App

Create repository

2 Phase 2: Create Supabase Project

Supabase

New Project

Project Name: TribeShop

Database Password: ••••••••••

Region: (Closest to you)

Create new project

3 Phase 3: Get Keys & Connection

Get the following from Supabase Project Settings:

- ✓ Project URL
- ✓ Anon Key
- ✓ Service Role Key
- ✓ Pooler Connection String (Use this for DATABASE_URL)

4 Phase 4: Antigravity Prompt

- Paste the full production prompt for **TribeShop**
- Tech Stack:

React · Node.js · PostgreSQL · Supabase

Frontend · Backend

5 Phase 5: Approve Antigravity

Antigravity is about to:

- ✓ Create folders & files
- ✓ Install dependencies
- ✓ Configure project
- ✓ Set up backend & frontend

Approve | Cancel

6 Phase 6: Configure Backend

backend/.env

```
PORT=5000
DATABASE_URL="postgresql://..."
DB_SSL=true
NODE_ENV=development
SUPABASE_URL=...
SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...
```

IMPORTANT: Use Supabase Pooler

7 Phase 7: Initialize Database

TERMINAL

```
$ cd backend
$ npm run db:init
```

✓ Database initialized successfully!
Tables, migrations & seed data created.

8 Phase 8: Start Backend

TERMINAL

```
$ cd backend
$ npm install
$ npm run dev
```

✓ Server running on http://localhost:5000

9 Phase 9: Verify Backend

http://localhost:5000/health

```
{
  "status": "OK",
  "message": "TribeShop API is running"
}
```

✓ Health check successful!

10 Phase 10: Setup Frontend

TERMINAL

```
$ cd frontend
$ npm install
$ npm run dev
```

Local: http://localhost:5173

11 Phase 11: Environment Variables

frontend/.env

```
VITE_API_BASE_URL=http://localhost:5000/api
VITE_SUPABASE_URL=...
VITE_SUPABASE_ANON_KEY=...
```

✓ Restart frontend server

12 Phase 12: Test Application

TribeShop | Home | Products | Cart

Welcome to TribeShop

Shop Now

✓ Frontend connected to backend!

13 Phase 13: Admin Login

Admin Login

Email: admin@tribeshop.com

Password: ••••••••

Login

Default: admin@tribeshop.com
Password: admin123

14 Phase 14: Deploy Backend

Render

- ✓ Connect GitHub Repo
- ✓ Set Environment Variables
- ✓ Build Command: **npm install**
- ✓ Start Command: **npm start**

Backend URL: https://api-tribeshop.onrender.com

15 Phase 15: Deploy Frontend

Netlify

- ✓ Connect GitHub Repo
- ✓ Build Command: **npm run build**
- ✓ Publish Directory: **dist**

Frontend URL: https://tribeshop.netlify.app

16 Phase 16: Update Frontend Environment

frontend/.env (Production)

```
VITE_API_BASE_URL= https://api-tribeshop.onrender.com/api
VITE_SUPABASE_URL=....
VITE_SUPABASE_ANON_KEY=...
```

Rebuild & Redeploy on Netlify

17 Phase 17: Final Verification

- ✓ Frontend Live
- ✓ Backend Live
- ✓ Admin Panel Working
- ✓ Products & Orders Working
- ✓ Database Connected



TribeSh

Your full stack ap

You've successfully built, deployed, and launched your full stack e-commerce app – TribeShop!

TribeShop: The End-to-End Playbook

Phase 0: Create Accounts

Create your accounts on:

- GitHub: https://github.com/
- Supabase: https://supabase.com/
- Render: https://render.com/
- Netlify: https://app.netlify.com/
- Antigravity: https://antigravity.google/product/antigravity-ide

Download:

- Antigravity IDE: https://antigravity.google/product/antigravity-ide
- Github Desktop: https://desktop.github.com/download/

Do this before opening Antigravity.

Phase 1: Create Repository

GitHub → New Repository

Repository name:

```
tribeshop
```

Visibility:

```
Private (recommended)
```

Phase 2: Create Supabase Project

Supabase Dashboard: New Project

Project Name: TribeShop

Database Password: Use only letters, numbers, underscores.

Example: Salonipassword1_23

Choose region closest to you.

Phase 4: Antigravity Prompt

Open Antigravity

Create folder : TribeShop

Paste the prompt!

Build a production-ready ecommerce application called TribeShop.

Frontend:

- React
- Vite
- React Router
- Context API
- Axios
- Responsive design

Backend:

- Node.js
- Express
- PostgreSQL
- Supabase
- REST API

Features:
Customer:

- Home
- Product listing
- Product detail
- Cart
- Checkout
- Order success

Admin:

- Login
- Dashboard
- Product CRUD
- Orders

Include:

- Production-ready folder structure
- Error handling
- Input validation
- Loading states
- README
- Health endpoints
- Environment validation
- API documentation
- Default admin credentials
- Database migration
- Seed scripts
- db:init command
- db:seed command

Assume developer starts with a

Phase 5: Approve Antigravity

Review changes.

Accept.

Verify structure:

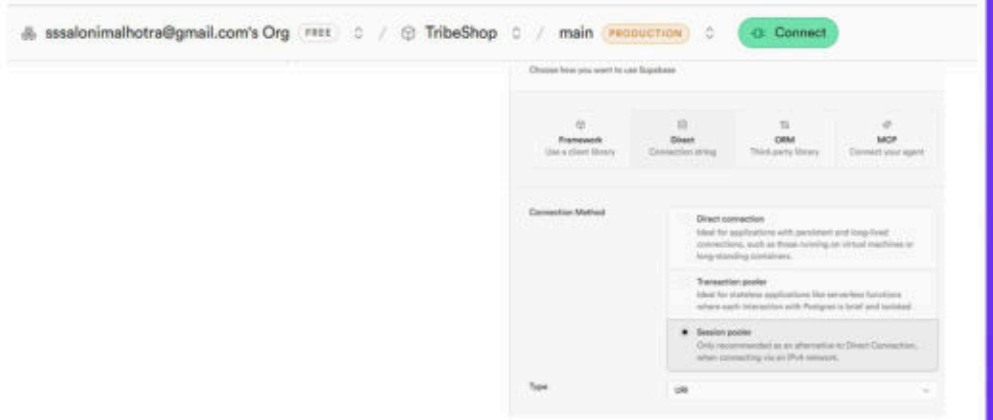
```
tribeshop/
├── frontend/
├── backend/
├── README.md
└── .gitignore
```

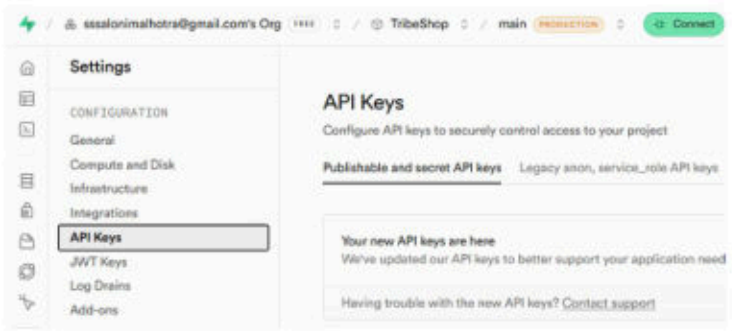
Phase 6: Configure Backend

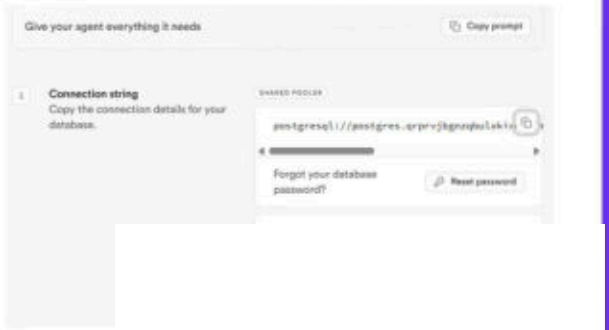
Open the .env file under backend/

```
PORT=5000
DATABASE_URL="POOLER_URL"
DB_SSL=true
NODE_ENV=development
SUPABASE_URL=...
SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...
```

IMPORTANT: Use Supabase Pooler







Phase 7: Initialize Database

Open your terminal in Antigravity

```
cd backend
npm run db:init
```

Phase 8: Start Backend

cd backend

npm install

npm run dev

Phase 9: Start Frontend

Open new terminal

cd frontend

npm install

npm run dev

Open http://localhost:5173

Phase 10: Verify Locally

Check if everything works!

Username: admin

Password: adminpassword123

Phase 11: GitHub

Database Password: Use only letters, numbers, underscores.

Example: Salonipassword1_23

Phase 11: GitHub

```
git init

git add .

git commit -m "Initial TribeShop"

git push -u origin master
```

Phase 12: Deploy Backend (Render)

1. New > Web service
2. Select: sssaloni/tribeshop
3. Set:Root Directory: backend
4. Build: npm install
5. Start : npm start
6. Environment Variables: Paste all backend env vars.
7. Deploy.

Phase 13: Verify Render

Root URL:

https://tribeshop.onrender.com

https://tribeshop.onrender.com/api/products
Should return json in browser

May return:

JSON

{"success":false,"message":"Resource not found"}

THIS IS GOOD.

Backend is running.

Phase 14: Deploy Frontend (Netlify)

Netlify:

Add New Site
Import from GitHub

Choose: tribeshop

Settings:

1. Base Directory:frontend
2. Build:npm run build
3. Publish: dist
4. Deploy

Set Environment variables:

VITE_API_URL: https://tribeshop.onrender.com/api

Phase 15: Redeploy Netlify

VERY IMPORTANT.

Environment variables do not apply retroactively.

After changing them:

```
Deploys
Trigger Deploy
Deploy Site
```

Phase 16: If redirect issues

Create: frontend/public/_redirects

Add the following:

/* /index.html 200

```
git add .
git commit -m "Add Netlify redirects"
git push
```

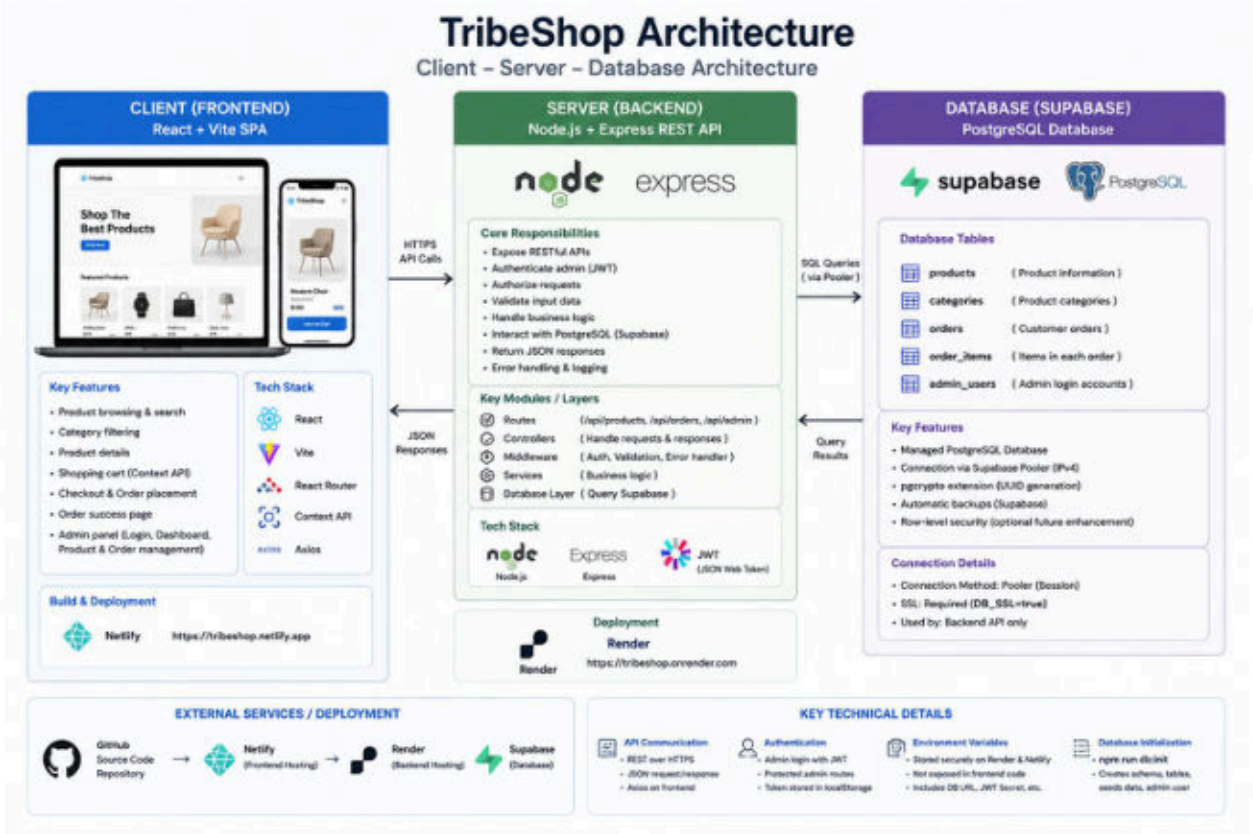
Netlify redeploys on its own.

Phase 17: Test in Prod

Check if everything works fine!

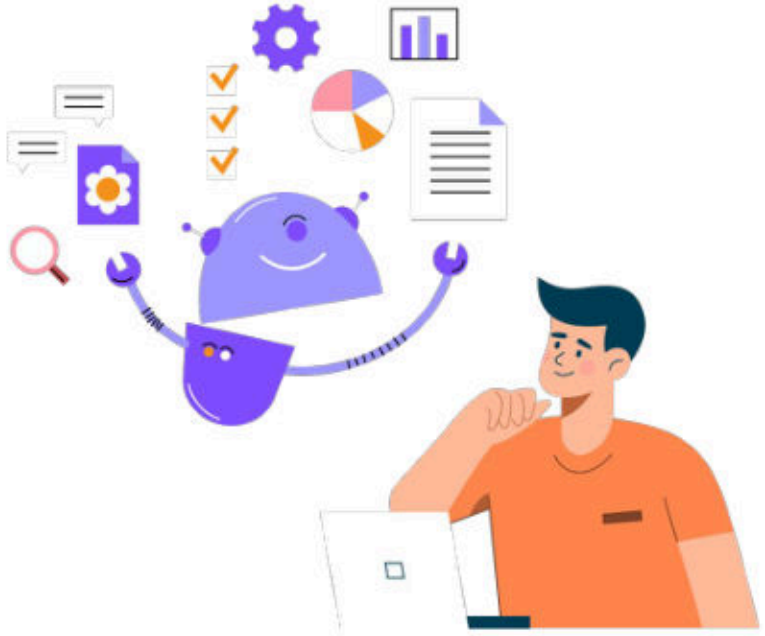
Detailed Architecture

https://github.com/sssaloni/tribeshop



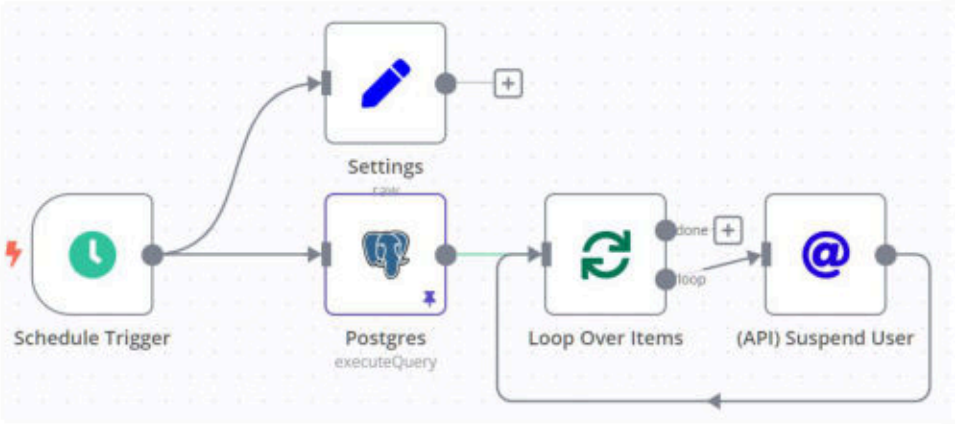
Why Automation Matters

- Reduce repetitive work
- Standardize PM routines (feedback, alerts)
- Prototype product logic quickly

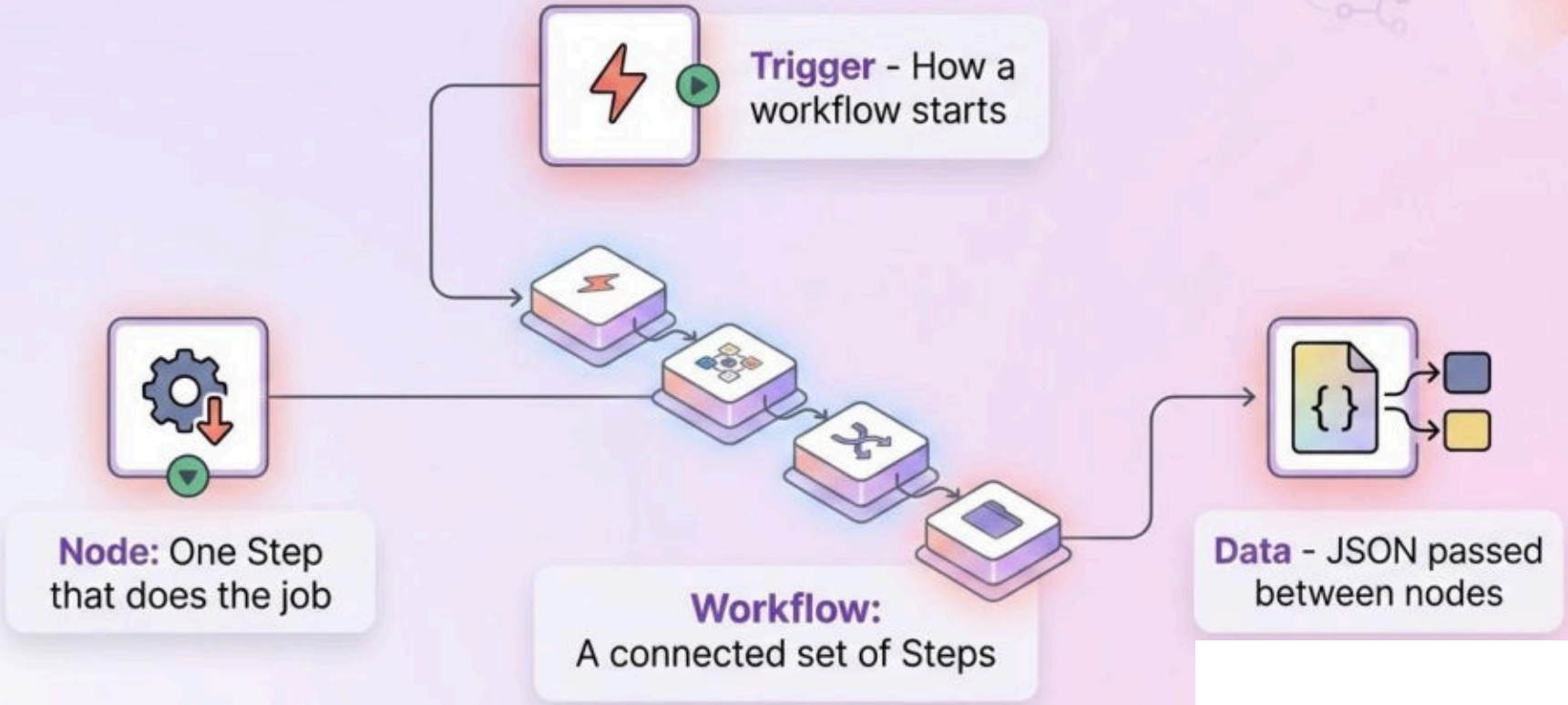


Meet n8n — Your Automation Workbench

- Open-source no-code automation tool
- Node-based flow builder
- Integrates with APIs & AI models



Core n8n Concepts



Core Concepts

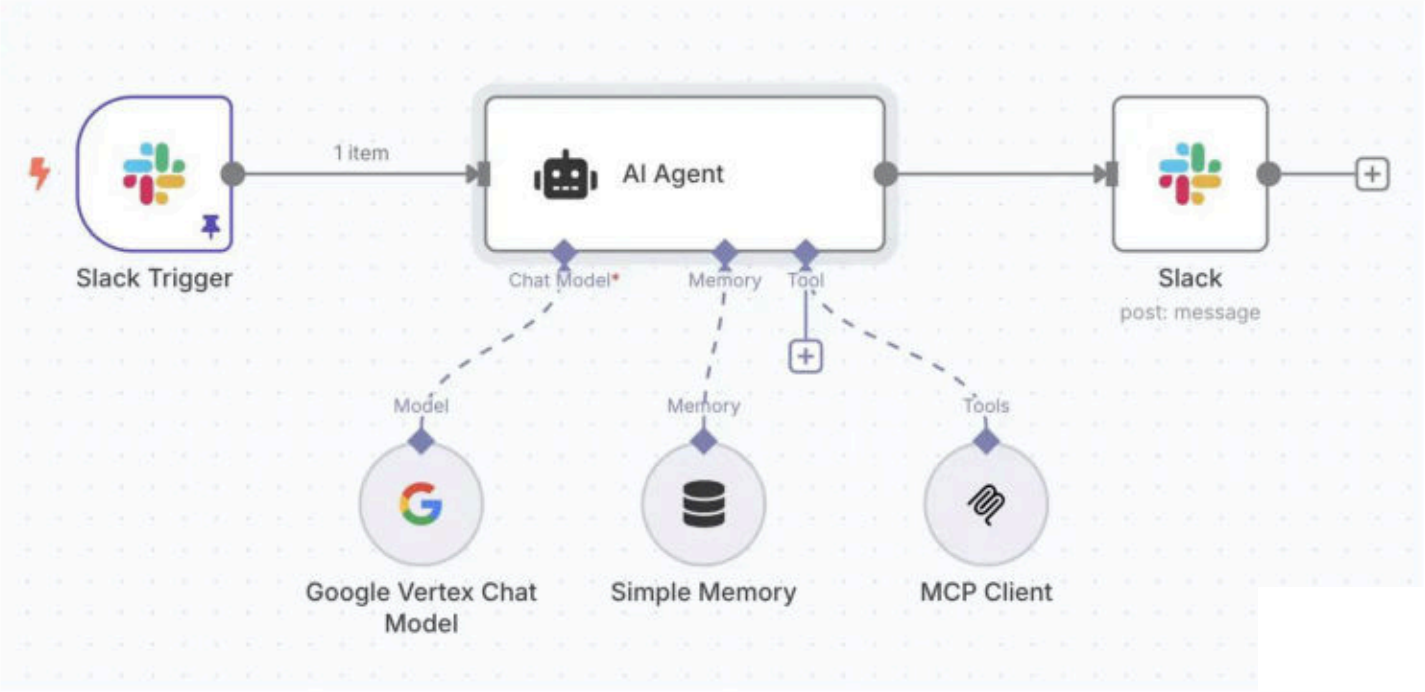
TERM	MEANING	PM ANALOGY
Trigger	Starts the workflow	“Event happens”
Node	One action step	“Task in your flow”
Workflow	Full sequence	“Process blueprint”

Understanding JSON, APIs & Webhooks

- JSON is structured data (Key-value pair)
  - Each node outputs JSON
  - Next node consumes that JSON
- APIs
  - API is application's interface (e.g. A waiter in a restaurant connecting your needs with the Kitchen)
  - APIs send and Receive JSON
- Webhooks
  - Used when system pushes events (e.g. In self Service restaurants , an announcement is made when your food is ready)
  - Example - Form submission -> Webhook

Simple Demo — Feedback to Slack

Flow: Google Form → Google Sheets → OpenAI → Slack.



AI Nodes in n8n

Key Idea: Add intelligence to your flows.

Examples:

- Text summary (OpenAI)
- Sentiment analysis (OpenAI/Claude)
- Content rewrite or generation

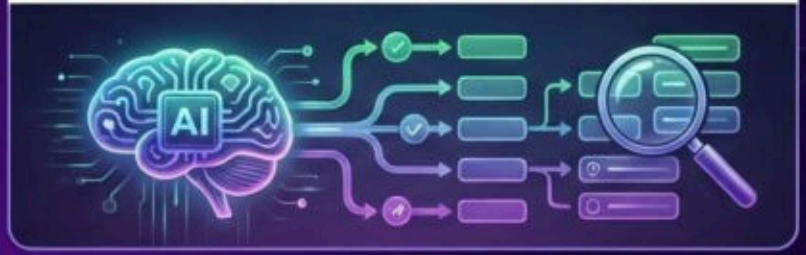


From Workflow → Agent

Workflow Automation

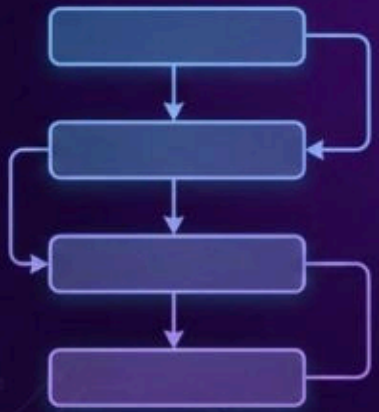


Agent

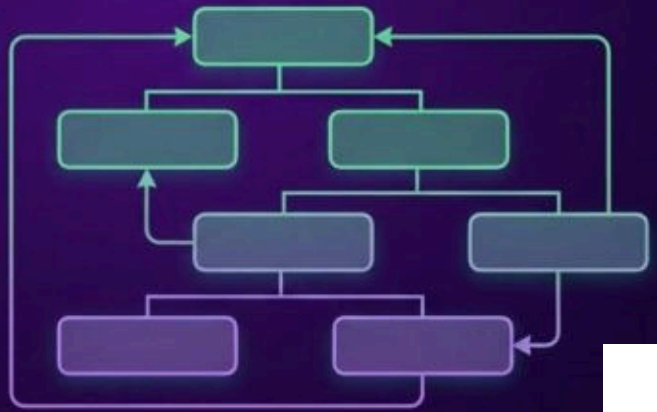


LINEAR

- Fixed steps, Think of it like a recipe,
- step 1-> step 2 -> Step 3 ,
- no decision making



LOOP



- Decides its own, what to do next.
- Think of it like hiring someone , give them a task and let them figure out the steps

Hands-On Challenge

Build these:

- Competitive Analysis
- Feature Request Tagger
- Lead/feature request classification and notification/update in backlog

Deliverable: Screenshot of n8n flow + short prompt used.

Session 1 Takeaways

- Understood automation vs agents
- Explored n8n nodes and AI integrations
- Built first intelligent workflow

Bridge: “Next session — we’ll monetize, measure and evaluate these systems.”

Scaling AI Systems — From Flows to Business Value

Because good automation should pay for itself.

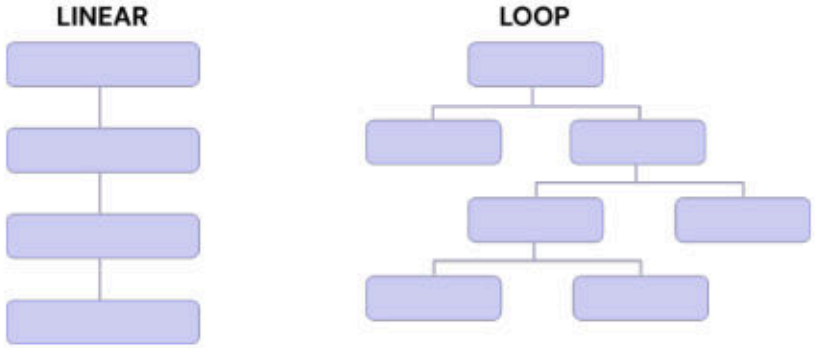


Day 2 :- Agenda

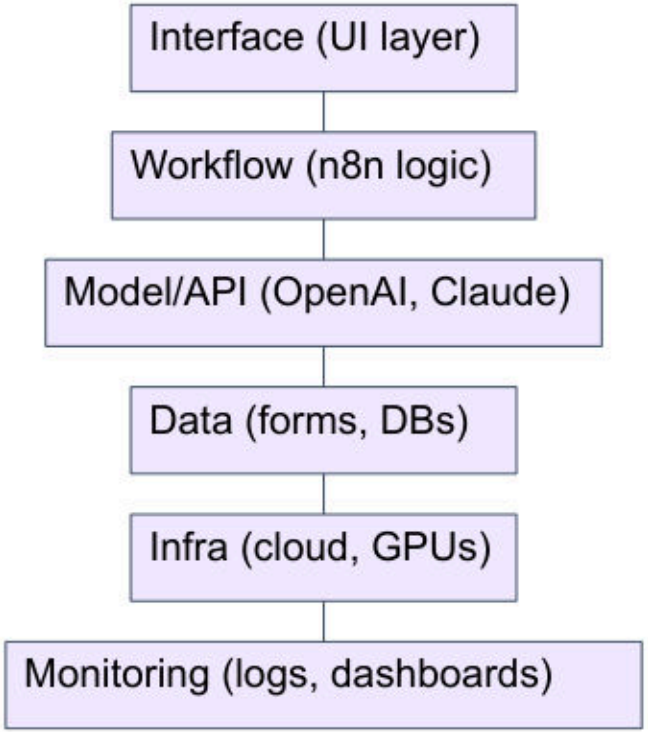
- Automation vs Agent (recap)
- Layers of an AI System
- AI Economics — APIs and Cost Drivers
- Monetization Patterns
- Real Case Studies (Jasper, Notion, Intercom)
- Evaluation Metrics
- Hands-on Task

Automation vs Agent

TYPE	DESCRIPTION	EXAMPLE
Automation	Fixed steps	Slack alert on NPS < 7
Agent	Decides what to do	Slack route by topic

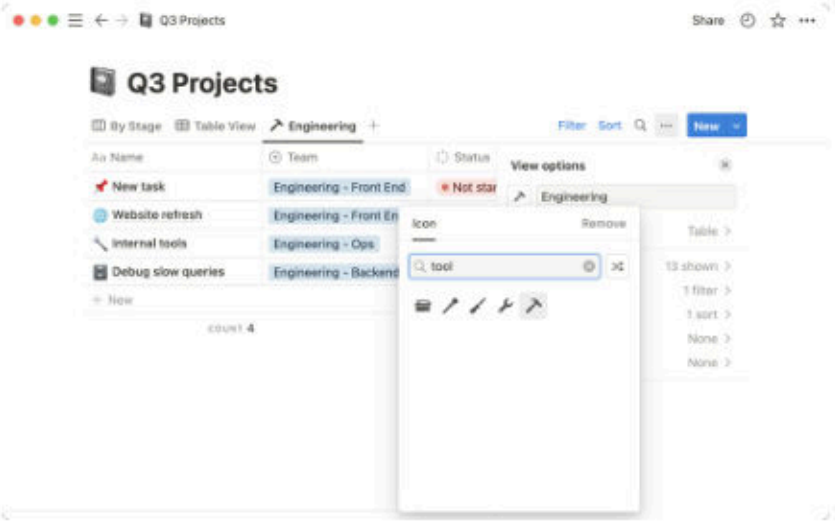


Layers of an AI System



Notion AI Stack

- Editor as UI
- Prompts → OpenAI
- Pricing: $10/month add-on
- Positioned as an embedded AI tier



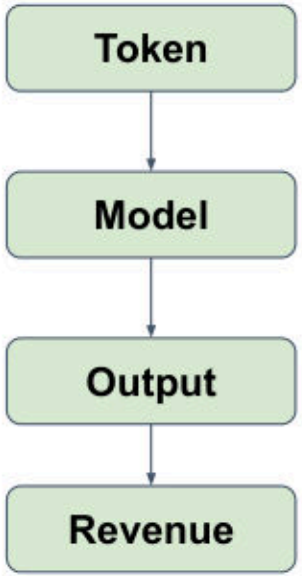
AI Economics Overview

API cost = price × tokens

Infra: Cloud + GPU usage

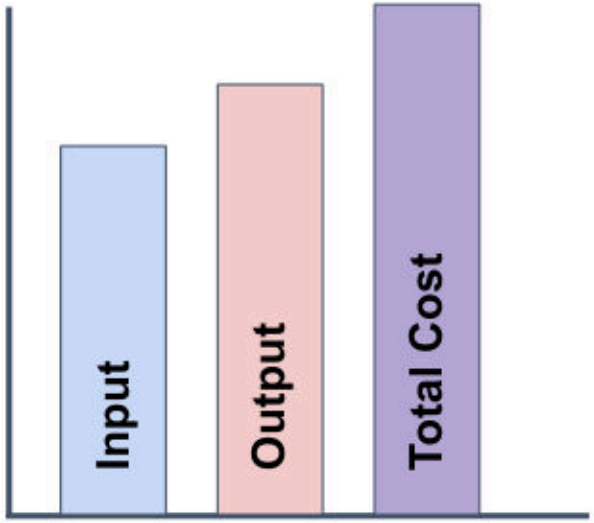
Margins vary with volume and reuse

Token → Model → Output → Revenue



Token Cost Breakdown

Type	Cost estimate
1M input tokens	$5
1M output tokens	$15
Total / 1M	~$20



Monetization Patterns : Turning AI into Revenue

Model	Example	Mechanism
Add-on	Notion AI	$10/month
Subscription	Jasper	$39/month
Per-action	Intercom Fin	$0.99/resolution

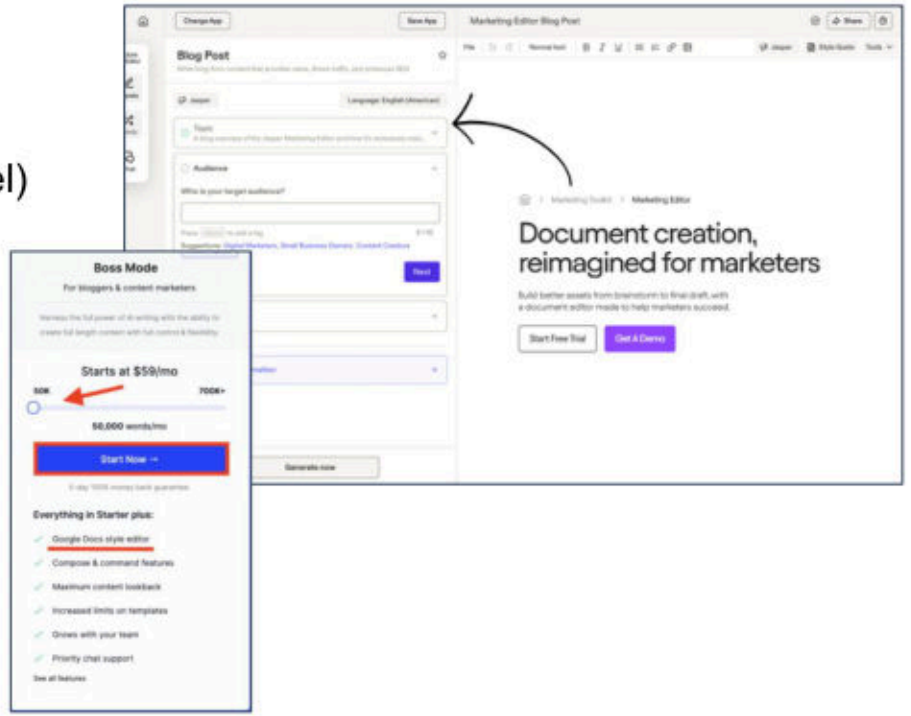




Fin AI

Jasper Case Study

- Target: Content marketers
- Product: AI content generator
- Pricing: $39/mo (template model)
- Revenue: $75M+ ARR



Intercom Fin Case

- Agent resolves tickets
- Pricing: $0.99 per resolution
- Tracks ROI per convo



Copilot vs Bundled AI

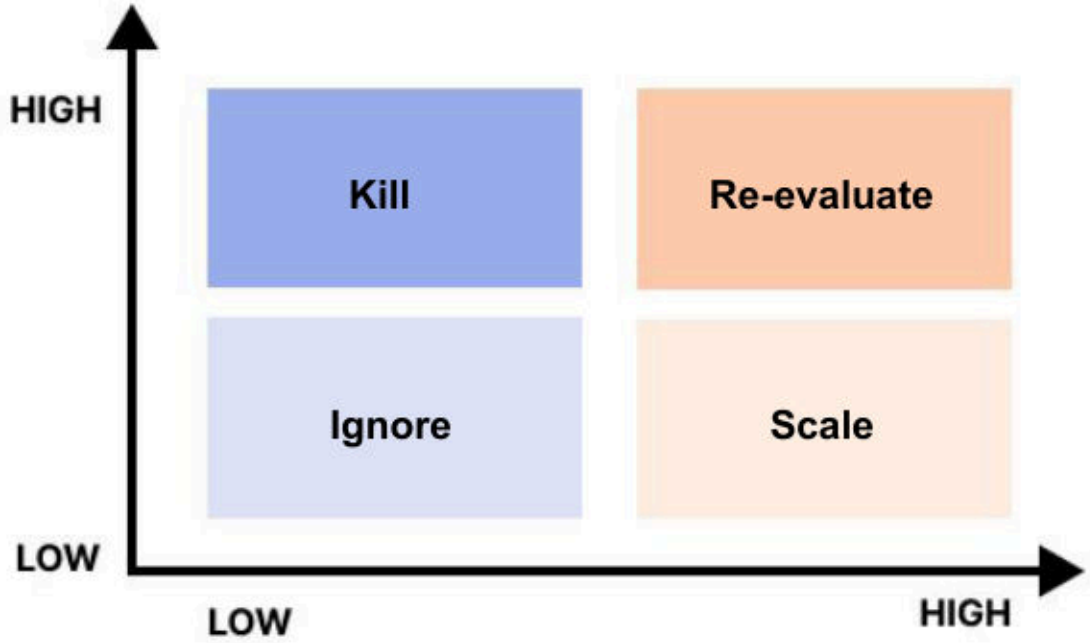
- Copilot: Paid AI assistant
- Shopify, Zoom: Free embedded AI

Rationale: Retention vs Revenue

Lenny Rachitsky's rule: If <70% users use it → charge separately. If >70% → bundle to drive stickiness.



Cost vs Value Matrix

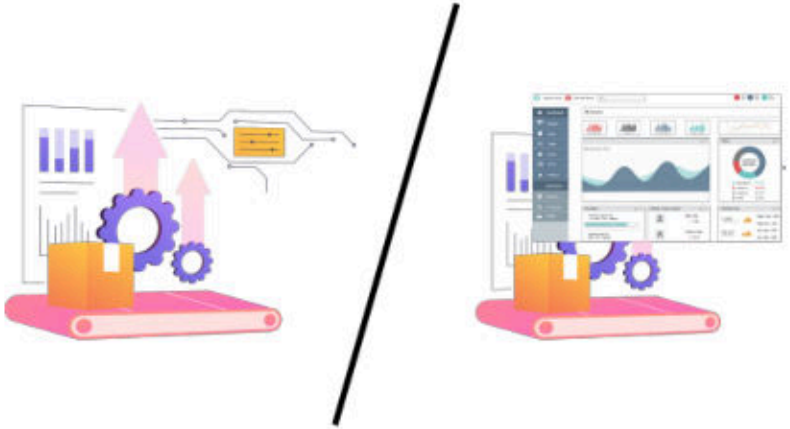


Evaluating AI Systems

Metric	How to Test	PM Action
Accuracy	10 sample outputs	Tune prompt / feedback
Latency	Avg response time	Optimise nodes
Cost	Token logs	Estimate ROI
Bias	Compare segments	Document risks
Hallucination	Fact check subset	Improve retrieval

Why Evaluation Matters

- “Automation ≠ Impact — Evaluation turns cool demos into scalable systems.”
- Why measure:
  - To link AI performance → business outcomes
  - To detect drift, errors, or waste
  - To justify ROI
- *‘What gets measured, improves.’ — Peter Drucker*



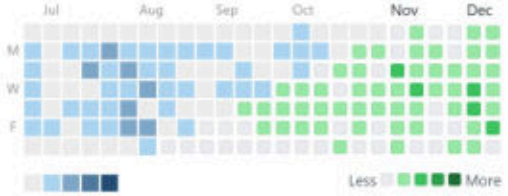
Case Study: Intercom Fin

- Intercom's **Fin AI bot** handles support conversations
- Pricing: **$0.99 per resolved conversation**
- Evaluation Metrics:
  - Containment rate (how many tickets AI resolved)
  - CSAT (Customer Satisfaction)
  - Cost per resolution
- AI effectiveness = measurable outcomes



Case Studies: GitHub Copilot & Jasper AI

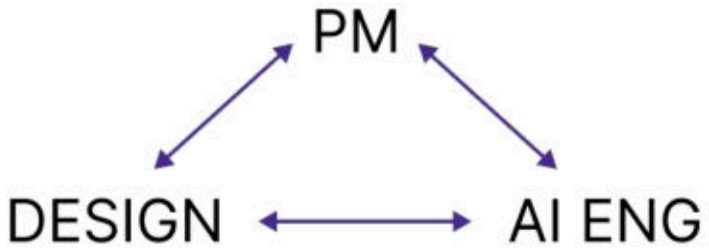
Product	Key Metric	Impact
GitHub Copilot	55% faster code completion	40% fewer coding errors
Jasper AI	10× faster content generation	90% brand-tone match





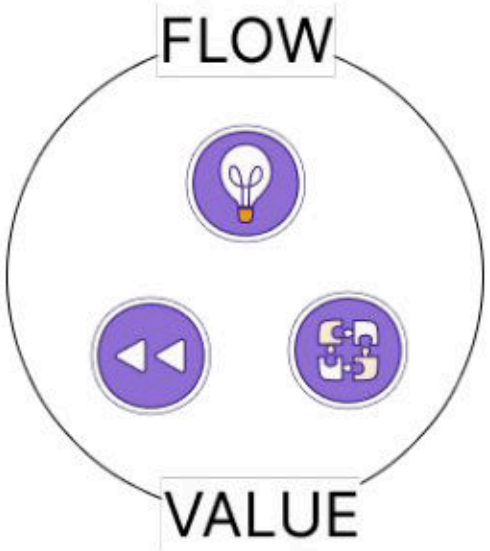
Collaborating with AI/ML Teams

- Define input/output early
- Share prompts with devs
- Co-track logs and cost



Wrap-Up & Reflection

- Understood AI economics & monetisation
- Connected flows to real business metrics
- Logged costs and accuracy
- Collaborated like a PM with AI teams



Bridge:

“Next — Capstone Project :- Where you can build you own AI powered product end to end.



Thank you!

MODULE 12 · FURTHER READING

Sources and further reading: Agentic AI and AI systems

How to read this page. These sources were found by web search and are third-party unless marked otherwise. Each note describes what the source says. Check the page before you quote any figure. We could not find NextLeap's own curriculum for this topic on the web, so no NextLeap syllabus is claimed here.

McKinsey: Where AI agents pay off
https://www.mckinsey.com/capabilities/quantumblack/our-insights/where-AI-agents-pay-off-a-practical-guide-to-the-economics-of-agentic-workflows

A practical guide to agentic economics: workflows need enough scale and enough automation potential for value to build up.

Tianpan: The unit economics of AI agents
https://tianpan.co/blog/2026-04-10-unit-economics-ai-agents-when-autonomous-work-saves-money

Argues for comparing the fully loaded cost of an agent with the fully loaded cost of the work it replaces, and for counting retries and variance.

JumpCloud: Agent unit economics vs standard AI pricing
https://jumpcloud.com/it-index/agent-unit-economics-vs-standard-ai-pricing

Compares per-agent economics with standard AI pricing.

Token cost guide: AI unit economics (cc.bruniaux.com)
https://cc.bruniaux.com/guide/ai-unit-economics/

A general primer on AI unit economics.